

Integrando Plataformas de Nuvens a Federações de Identidade^{1,2}

Ioram S. Sette^{1,II}, Carlos A. G. Ferraz^I

^I Centro de Informática – Universidade Federal de Pernambuco (UFPE) – Recife - PE

^{II} Centro de Estudos e Sistemas Avançados do Recife (CESAR) – Recife – PE

{iss,cagf}@cin.ufpe.br

Abstract. *Privacy of the data processed and stored in the cloud is a concern for the users of these services. Cloud platforms are exposed at the Internet, their usage is shared with other users, and third parties manage them. Identity and access control mechanisms are relevant for intending to protect data from improper access. Among them, identity federations let the user's authentication to be performed by entities which are closer to it. The proposal of this work is to integrate Openstack cloud platform, through its authentication module Keystone, to identity federations using SAML and OpenID Connect protocols. These protocols are compared observing the ease of integration and performance.*

Resumo. *A privacidade dos dados processados e armazenados em nuvens de computadores é uma preocupação para os usuários destes serviços. Plataformas de nuvens estão expostas na Internet, seu uso é compartilhado com outros usuários e são gerenciadas por terceiros. Mecanismos de controle de identidade e acesso são importantes para proteger as informações de acessos indevidos. Dentre eles, as federações de identidade permitem que a autenticação dos usuários seja realizada por entidades mais próximas aos mesmos. A proposta deste trabalho é integrar a plataforma de nuvem Openstack, através de seu módulo de autenticação Keystone, a federações de identidade usando os protocolos SAML e OpenID Connect. Estes protocolos são comparados com relação à facilidade de integração e desempenho.*

1. Introdução

O uso de nuvens de computadores por usuários e empresas vem crescendo a cada dia. Usuários usam serviços de nuvem em suas modalidades IaaS (*Infrastructure as a Service*), PaaS (*Platform as a Service*) e SaaS (*Software as a Service*), enviando seus dados privados para serem processados e armazenados nestas plataformas [Columbus 2013a; Columbus 2013b]. A autenticação e autorização dos usuários através de mecanismos de controle de acesso e de uso aos serviços e dados hospedados em plataformas de nuvens são importantes para evitar o uso indevido e a quebra da privacidade [Tavisi *et al* 2012; Karp *et al* 2009].

Federações de identidade permitem que a autenticação dos usuários seja realizada por entidades que possuem relacionamento próximo aos mesmos, por exemplo: universidade-alunos/professores/pesquisadores, empresas-colaboradores, banco-cliente, grandes provedores de serviço na Internet-usuários. Estas instituições por muitas vezes podem utilizar mecanismos sofisticados de autenticação como biometria, *smartcards*, *tokens*, ou uma combinação deles (autenticação multi-fator). O uso de federações também permite que o usuário se autentique uma única vez e obtenha acesso a vários serviços e sistemas diferentes através do mecanismo de *Single Sign-On* (SSO). Por fim, ao centralizar a autenticação, os usuários passam a ter menos senhas para lembrar, podendo escolher senhas mais fortes e seguras [Revar *et al* 2011].

¹ Projeto apoiado pela Rede Nacional de Pesquisa (RNP) - Edital PGId 2013.

² Este trabalho foi apoiado em parte pelo Instituto Nacional de Ciência e Tecnologia para Engenharia de Software (INES), financiado pelo CNPq e FACEPE, processos 573964/2008-4 e APQ-1037-1.03/08.

Dentre os protocolos de federação de identidade mais relevantes encontram-se o SAML (*Security Assertion Markup Language*) e OpenID Connect. Estes protocolos permitem a troca de dados de autenticação e autorização entre provedores de identidade (IdP – *Identity Provider*) e provedores de serviço (SP – *Service Provider*). O SAML (<https://www.oasis-open.org/committees/security/>) é bastante usado no meio acadêmico e por empresas que possuem uma relação de confiança entre si. Trata-se de um protocolo aberto baseado em XML. O OpenID Connect (<http://openid.net/connect>) também é um protocolo aberto, baseado em REST/JSON. Ele é mais recente e usado por grandes provedores de serviço na Internet, como Microsoft e Google [Lynch 2011].

Os principais provedores de computação em nuvem desenvolveram suas próprias plataformas, padrões e tecnologias. Em contrapartida, a comunidade de software livre conta com alguns projetos abertos de plataformas nos modelos IaaS (ex. Openstack e Eucalyptus) e PaaS (ex. Apache Hadoop). Plataformas de IaaS possuem em suas arquiteturas um módulo responsável pela gestão de identidade e acesso. Este módulo se integra aos demais, oferecendo a eles serviço de autenticação e autorização. Nas versões mais recentes das plataformas Eucalyptus (2013b) e Openstack (2013a), os módulos de gestão de identidade e acesso são capazes de se integrar com sistemas de diretório externos, por exemplo, através do protocolo LDAP (*Lightweight Directory Access Protocol*). Porém, apesar de permitirem extensões, eles não oferecem em suas distribuições oficiais mecanismos para integração com federações de identidade [Eucalyptus 2013b; Openstack 2013 p.109-113].

Neste trabalho, a plataforma Openstack, apoiada por mais de 200 empresas (ex. HP, IBM e Cisco) e adotada pelo GT-CNC (Computação em Nuvem para Ciência) da RNP [Diniz *et al* 2013; Silva *et al* 2013], é estendida para suportar autenticação através de federações de identidade utilizando o protocolo OpenID Connect. A versão adotada será a de Chadwick (2013a), que já se integra com provedores de identidade através de SAML. Desta forma, será possível comparar os protocolos de federação de identidade quanto à facilidade de integração com o Openstack e, uma vez integrados a esta plataforma, comparar os desempenhos dos referidos protocolos.

Este artigo se divide em mais 5 seções. A seção 2 discute os trabalhos relacionados. Um modelo para autenticação e autorização em plataformas de nuvem é discutido na seção 3. A seção 4 descreve como funciona a integração da plataforma Openstack com federações de identidade. A seção 5 compara os protocolos SAML e OpenID Connect em relação à integração com o Openstack. A seção 6 descreve os experimentos realizados e avalia os resultados. Por fim, a seção 7 apresenta as conclusões e trabalhos futuros.

2. Trabalhos Relacionados

Os trabalhos a seguir abordam a integração entre plataformas de nuvem e federações de identidade.

Um protótipo com intenção de integrar a plataforma Openstack com provedores de identidade utilizando o protocolo OpenID foi construído por Khan *et al* (2011). No entanto, a versão do Openstack utilizada na época não possuía um módulo de autenticação bem definido, como o Keystone presente nas versões atuais. Então, a API de serviços do Openstack foi alterada criando-se funções para autenticação através do protocolo OpenID, versão inicial que inspirou a criação do OpenID Connect.

Revar *et al* (2011) descrevem uma arquitetura de alto nível e trechos de códigos mostrando que é possível implementar um mecanismo de *Single Sign-On* utilizando infraestrutura de chaves públicas com certificados X.509, na plataforma Eucalyptus.

Em [Chadwick *et al* 2012], a plataforma Eucalyptus é integrada com provedores de identidade através do protocolo SAML. Em [Chadwick *et al* 2013], o mesmo autor integra a plataforma Openstack, em versão atual com o módulo de identidade (Keystone), com provedores de identidade também usando o protocolo SAML. O IdP utilizado foi o

SimpleSAMLphp, que além de implementar o protocolo SAML possui um componente chamado “Proxy IdP” que faz interface com outros provedores de identidade nos padrões SAML, OAuth, OpenID, dentre outros. Em [Chadwick 2013a], o mesmo autor explica como funciona a integração do Keystone com federações de identidade e define dois fluxos de autenticação, um para cliente simples e um outro para cliente inteligente, protegido contra ataques de *phishing*.

Diniz *et al* (2013) estendem a solução de Chadwick (2013a) para integrar o Openstack a um IdP Shibboleth, apresentando as dificuldades e soluções encontradas. Em [Silva *et al* 2013], os autores apresentam um estudo de caso para integração do serviço Swift do Openstack com autenticação federada através de IdP Shibboleth usando a solução de [Diniz *et al* 2013].

Resultados preliminares do nosso trabalho são apresentados em [Sette *et al* 2013]. A implementação de referência proposta em [Chadwick 2013a] é estendida para possibilitar que o “Keystone Federado” dê suporte ao protocolo OpenID Connect, sem a necessidade de *proxy* intermediário. Os resultados são apresentados na forma de uma avaliação de desempenho e comparações entre os protocolos SAML e OpenID Connect com perfil de cliente básico.

Neste artigo, o perfil de cliente implícito do OpenID Connect foi implementado no “Keystone Federado” e as análises e comparações adicionadas aos resultados obtidos em [Sette *et al* 2013]. Comparações mais aprofundadas entre os protocolos de federação de identidade SAML e OpenID Connect e uma análise sobre os mecanismos de autenticação e autorização em plataformas de nuvem são apresentadas, o que nenhum dos trabalhos relacionados aqui faz.

3. Autenticação e Autorização em Plataformas Abertas de IaaS

É comum, nas plataformas abertas de IaaS como Openstack e Eucalyptus, a existência de módulos dedicados a gestão da identidade e acesso dos usuários. Estes módulos costumam ser peças centrais e importantes nas arquiteturas, fazendo interface com todos os demais módulos.

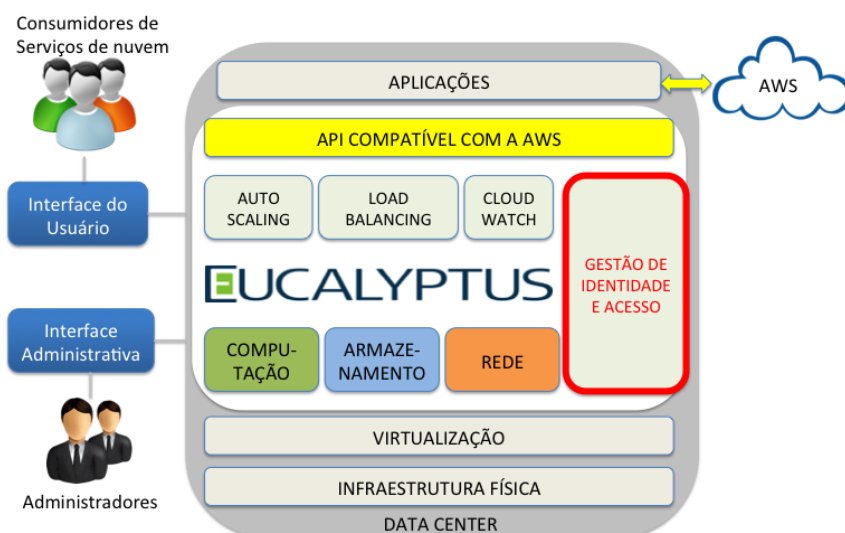


Figura 1. Arquitetura de alto nível do Eucalyptus - adaptado de [Eucalyptus 2013a]

No Eucalyptus, o módulo de Gestão de Identidade e Acesso, em destaque na Figura 1, tem esta função. Após a autenticação, regras (*políticas*) são verificadas a cada tentativa de acesso, de forma centralizada. A solução é bastante flexível, inspirada na AWS (*Amazon Web Services*). Neste modelo, é possível criar regras refinadas baseadas nos atributos dos recursos (ABAC – *Attribute-Based Access Control*). Por exemplo, pode-se criar uma regra

que permite o acesso de leitura apenas a arquivos menores que 1MB para um determinado grupo de usuários [Eucalyptus 2013a; Eucalyptus 2013b].

O Openstack (Figura 2) também possui um módulo de identidade responsável pela autenticação, o Keystone (em destaque). Além da autenticação, consideramos que o Keystone faz uma autorização de “alto nível”, ou seja, transforma os atributos de autenticação em papéis (RBAC - *Role-Based Access Control*). A autorização de fato ocorre posteriormente por cada módulo do Openstack de forma descentralizada com base nos papéis (*roles*), identificadores do usuário e projetos (*tenants*) ao qual o usuário faz parte. Desta forma, o Keystone se integra a todos os módulos da plataforma Openstack. Por exemplo, o módulo de armazenamento (Swift), baseado na identificação do usuário, do papel e do projeto, decide se libera as ações de *upload* e *download* de arquivos em determinada pasta. Ao contrário da solução Eucalyptus/AWS, esta solução não permite regras mais refinadas [Openstack 2013 p.3-4; Openstack 2013 p.109-113].

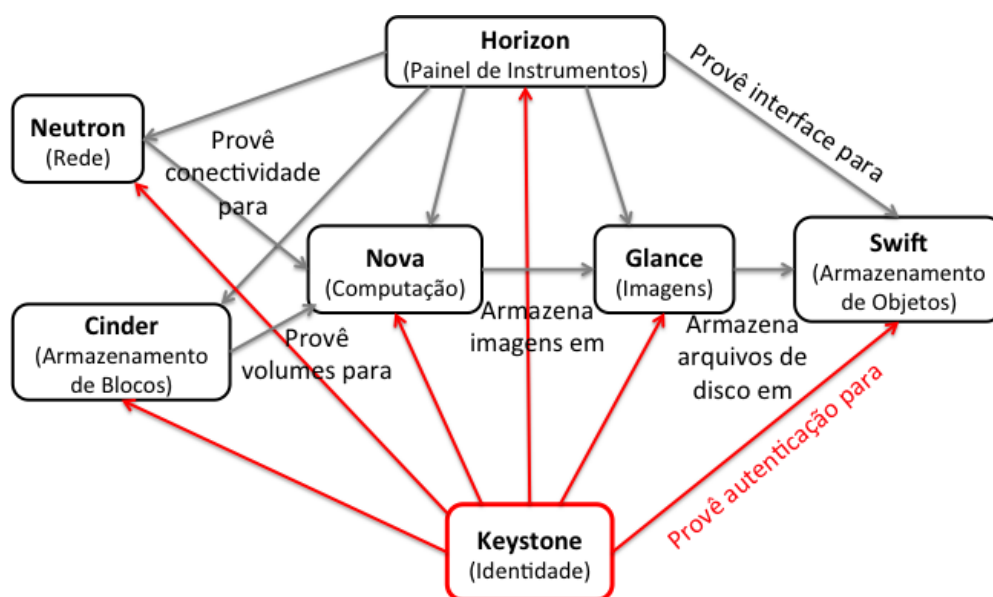


Figura 2. Arquitetura de alto nível do Openstack – adaptado de [Openstack 2013 p.3-4]

Apesar de ambas as soluções possuírem integração com serviços de diretórios através de consultas LDAP, nenhuma delas possuem nativamente integração com provedores de identidade externos, como os existentes numa federação de identidade.

Provedores de identidade podem fornecer informações valiosas sobre os usuários autenticados, como informações pessoais, acadêmicas, profissionais, culturais, dentre outras. As plataformas de nuvem utilizam estes atributos dos usuários em seus mecanismos de autorização. Regras de autorização (*policies*) são definidas nos mecanismos de controle de acesso como o ABAC e RBAC. No ABAC, o uso dos atributos é natural, podendo-se criar regras refinadas, por exemplo: apenas usuários de idade superior a 18 anos (atributo do usuário) pode assistir a filmes do gênero “violência” (atributo do recurso). Já no RBAC, os atributos são mapeados em papéis.

4. Plataforma Openstack Federada

A integração do Keystone com federações de identidade permite que uma relação de confiança seja estabelecida entre o Openstack, que atua como provedor de serviço (SP), e provedores de identidade (IdPs), onde os usuários são autenticados.

Desta forma, os provedores de identidade informam os atributos dos usuários autenticados ao Keystone, que os mapeia em um papel (*role*) e projeto (*tenant*) necessários para a autorização de acesso. Como o Keystone não conhece previamente todos os usuários

cadastrados nos IdPs, um identificador é gerado para cada usuário em cada primeiro acesso. Estes identificadores permitem a criação de regras específicas para determinado usuário.

A Figura 3 descreve o fluxo para uso de serviços da nuvem Openstack através de autenticação por IdPs federados. Em nosso estudo, usamos o serviço de armazenamento de dados (Swift) que permite o envio e recebimento de arquivos na nuvem. Para acessar o Swift, a ferramenta de linha de comando (CLI) chamada “cliente Swift” foi modificada para suportar federação de identidade. Para interagir com IdPs, os usuários utilizam navegadores web (*browsers*).

Quando o usuário realiza uma operação através do “cliente Swift”, a ferramenta inicialmente requisita um *token* ao serviço Keystone (1). Na versão não federada, o Keystone realiza a autenticação localmente, checando as credenciais em banco de dados ou serviço de diretório. Já na versão federada, o Keystone solicita ao usuário que escolha um provedor de identidade com o qual o serviço de nuvem possui uma relação de confiança estabelecida, através de uma lista. Na figura 3, duas opções são apresentadas: o IdP SAML (UFRN) e o IdP OpenID Connect, ou OIDC (RNP GiD Lab), implementando os respectivos protocolos.

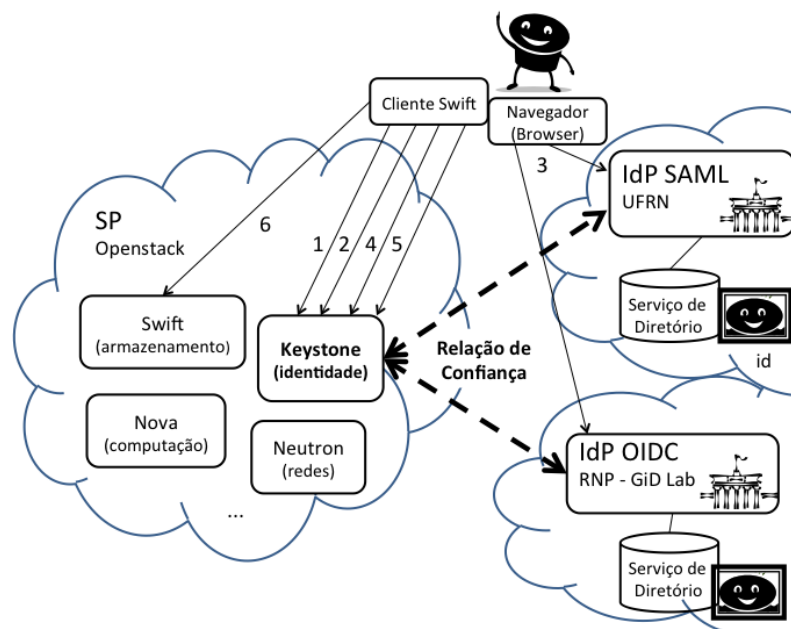


Figura 3. Fluxo de autenticação do Openstack Federado

Ao escolher um IdP (2) onde possui credenciais de acesso, o usuário se autentica por meio de um navegador web (3). Vários mecanismos de autenticação podem ser usados pelo IdP para autenticar os usuários, por exemplo, usuário/senha, *tokens* OTP (*one-time-password*), *smartcards*, biometria etc.

Ao final da autenticação, o IdP gera uma mensagem contendo o resultado da autenticação e os atributos do usuário e encaminha ao “cliente Swift”, que por sua vez entrega ao serviço Keystone (4). O Keystone valida a mensagem emitida pelo IdP e, baseado nos atributos do usuário, seleciona os projetos (*tenants*) e papéis que o usuário pode exercer em cada projeto. A lista de projetos é apresentada ao usuário, que deve escolher qual pretende utilizar. Por exemplo, apenas os usuários que se autenticam no IdP da UFRN podem acessar os serviços e recursos alocados na nuvem para esta instituição. De forma similar, existe o projeto RNP acessível apenas aos usuários autenticados pelos usuários do IdP da RNP. É possível ainda que exista um projeto público ou compartilhado entre instituições na nuvem. Neste caso, o projeto aceitaria o acesso de usuários autenticados em qualquer um dos dois IdPs.

Ao escolher o projeto (5), os papéis (*roles*) que o usuário possui para o projeto escolhido são definidos e um *token* é gerado e entregue ao usuário. Este *token* deve ser usado para acessar os serviços do Openstack. Um detalhe importante é que o usuário federado não precisa estar previamente cadastrado na nuvem Openstack para usar os seus serviços.

Por fim, o “cliente Swift” acessa o serviço Swift (6) solicitando a operação desejada, usando o *token* recebido. Um novo controle de acesso é realizado internamente pelo serviço Swift através de regras definidas em ACLs (*Access Control Lists*). Papéis administrativos podem ser definidos no Swift para dar aos usuários permissões plenas dentro do projeto, por exemplo: criar pastas, listar, enviar ou receber arquivos, além de criar as regras (ACL) que definem quais outros papéis ou usuários possuem que tipo de acesso (leitura/download/listagem ou escrita/upload) para cada pasta.

5. SAML e OpenID Connect

Nesta seção, uma comparação entre os protocolos SAML e OpenID Connect é realizada apontando as principais semelhanças e diferenças entre eles considerando o contexto da integração com o Openstack. Os fluxos de autenticação destes protocolos também são apresentados para cada protocolo e perfil testados nos experimentos realizados.

5.1. Comparações entre SAML e OpenID Connect

Do ponto de vista do usuário, os dois protocolos são bem semelhantes e realizam bem a proposta de autenticação SSO. A sequência de passos para os dois protocolos é exatamente a mesma: o SP redireciona o cliente para o IdP, que autentica o usuário e redireciona o cliente de volta ao SP passando dados que comprovam a autenticação do mesmo. Porém, nesta seção listaremos algumas diferenças que existem entre eles, resumidas na Tabela 1.

O protocolo SAML, criado em 2001, baseia-se na troca de mensagens de autenticação e autorização no padrão XML entre usuário e provedores de serviço e de identidade. É um protocolo maduro, com a sua última versão (2.0) lançada em 2005. Esta versão suporta vários mecanismos de transporte através de diferentes *bindings* [OASIS 2005a], por exemplo: SOAP (única opção na versão 1.x), HTTP Redirect (GET) e HTTP POST. O uso do HTTP sem o SOAP tornou o protocolo mais leve, apesar das mensagens ainda serem XML contendo certificados e assinaturas digitais. Além dos *bindings*, a nova versão permite diferentes fluxos de autenticação, ou *profiles* [OASIS 2005b]. No perfil *Web Browser SSO*, toda troca de dados entre IdP e SP passa pelo cliente através de mensagens assinadas, não havendo tráfego entre SP e IdP. Já no perfil *Artifact Resolution*, SP e IdP conversam diretamente para trocar dados de autenticação.

O OpenID Connect é mais recente (2012) e foi construído sobre o protocolo OAuth 2.0 [IETF 2012]. Trata-se de um protocolo RESTful transportando mensagens JSON entre usuário e provedores de serviço e de identidade. Apesar do pouco tempo, é utilizado por grandes empresas como Google, Yahoo, Facebook e Microsoft. Dois fluxos de autorização são definidos. O perfil de cliente básico é o mais usado e funciona de forma similar ao *Artifact Resolution* do SAML, onde SP e IdP trocam mensagens com dados da autenticação [Sakimura *et al* 2013a]. O perfil implícito [Sakimura *et al* 2013b] é um pouco mais leve, com menos mensagens trocadas entre SP e IdP, porém exige que o cliente (*browser*) seja alterado para enviar os dados enviados pelo IdP ao SP, uma vez que ele os envia através de fragmentos de URL.

A relação de confiança entre SP e IdP é estabelecida de diferentes formas entre os protocolos. No SAML, chaves públicas são trocadas previamente e usadas para assinar as mensagens trafegadas. Desta forma, o IdP pode verificar se a solicitação de autenticação veio de um SP confiável e o SP também pode conferir se a mensagem de autenticação foi emitida pelo IdP e se está íntegra. Isto possibilita a conversa entre SP e IdP por intermédio

do usuário. A opção de criptografia foi introduzida na versão 2.0. No OpenID Connect, o SP cadastra-se previamente no IdP e recebe um par de credenciais (*clientId* e *clientSecret*). Quando SP e IdP se comunicam o SP se autentica junto ao IdP através delas. Opcionalmente, as mensagens podem ser assinadas e/ou criptografadas pelo IdP através dos mecanismos JWS (*JSON Web Signature*) e JWE (*JSON Web Encryption*), respectivamente. Apesar de ambos os protocolos suportarem assinatura e criptografia em suas especificações, é prática comum o uso de assinaturas em IdPs SAML, mas não em IdPs OpenID Connect.

Os atributos dos usuários usados nas mensagens SAML são definidos em outras especificações, como X.500, LDAP, Internet2 eduPerson, ou esquemas privados baseados no perfil de atributo X.500/LDAP. No OpenID Connect, os atributos são agrupados em escopos (*scopes*). O único escopo obrigatório é o “openid” que possui o atributo (*claim*) “sub”, identificador do usuário no IdP. Os escopos opcionais são “profile”, “email”, “address” e “phone”, com atributos bem definidos na especificação. Escopos adicionais definidos pelo usuário também são permitidos.

Tabela 1. Resumo das comparações entre protocolos SAML e OpenID Connect

	SAML	OpenID Connect/OAuth2
Versão atual	2.0 (2005)	1.0 (2012) / 2.0 (2012)
Data da primeira versão	2001	2012 / 2006
Protocolo/Mensagens	Vários/XML	REST/JSON
Suporta mensagens assinadas?	Sim (XML Signature)	Sim (JWS)
Suporta mensagens criptografadas?	Sim (XML Encryption)	Sim (JWE)
Atributos	X.500/LDAP	Scopes/Claims
No. Interações Usuário x IdP	2	2
No. Mínimo de Interações SP x IdP	0 (perfil Web SSO)	1 (perfil cliente implícito)

Os experimentos deste trabalho comparam os protocolos SAML com *binding HTTP Redirect*, perfil *Web Browser SSO* (mais eficiente) e mensagens assinadas e OpenID Connect com perfis de usuário básico e implícito, sem assinatura ou criptografia.

5.2. Fluxos de Autenticação

Os diagramas de sequência a seguir apresentam os fluxos de autenticação através do Keystone não federado (Figura 4) e federado usando os protocolos SAML com perfil *Web Browser SSO* (Figura 5), OpenID Connect com perfis de cliente básico (Figura 6) e implícito (Figura 7), respectivamente. Nos exemplos, o usuário solicita a listagem dos arquivos em uma pasta no Swift.

O diagrama da Figura 4 apresenta o fluxo de autenticação do Openstack não federado. Ao usar o “cliente swift” neste modo, o usuário informa o projeto, credenciais do usuário e a pasta que deseja listar (1). O “cliente swift” solicita ao Keystone um *token* (1.1), usado para acessar o serviço Swift (1.2).

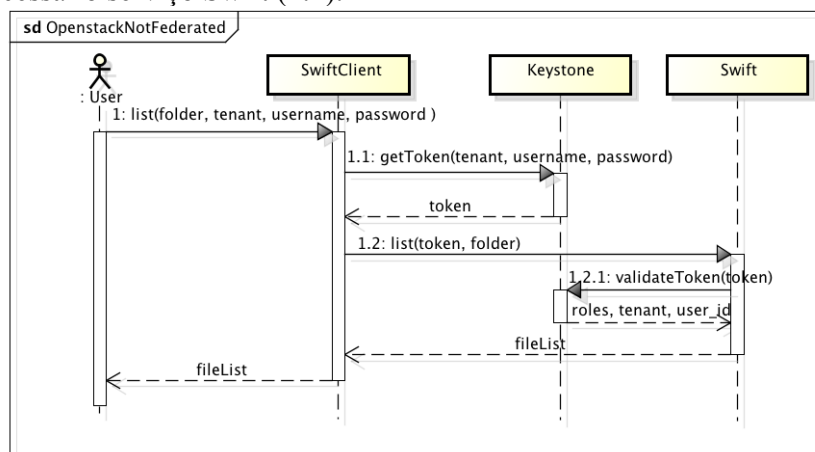


Figura 4. Fluxo de Autenticação do Openstack não federado

Na Figura 5, o diagrama apresenta o fluxo de autenticação do Openstack federado utilizando um IdP SAML com *binding* HTTP Redirect. Ao usar o “cliente swift”, o usuário informa apenas a pasta que deseja listar (1). O “cliente swift” solicita ao Keystone a lista de IdPs disponíveis para autenticação (1.1). O usuário escolhe o IdP (2) e o Keystone devolve a URL de autenticação (1.2). O “cliente swift” abre um browser chamando a URL (1.3) e o usuário se autentica no IdP (3). O IdP SAML responde com uma mensagem do tipo SAMLResponse, assinada pelo SP (3.2). O Keystone então valida a mensagem e descobre os atributos do usuário autenticado (1.4.1). Neste momento, um ID de usuário é gerado a partir do identificador principal do usuário e criado na base de usuários do Keystone, caso não exista. Em seguida, o Keystone usa um mapa entre atributos do IdP e do SP, e descobre quais os projetos que o usuário pode acessar (1.4.2). O usuário escolhe um projeto (4) e o “cliente swift” solicita um *token* válido para acessar o serviço Swift (1.5). Finalmente, o “cliente swift” solicita ao serviço Swift (1.6) a listagem da pasta usando o *token*.

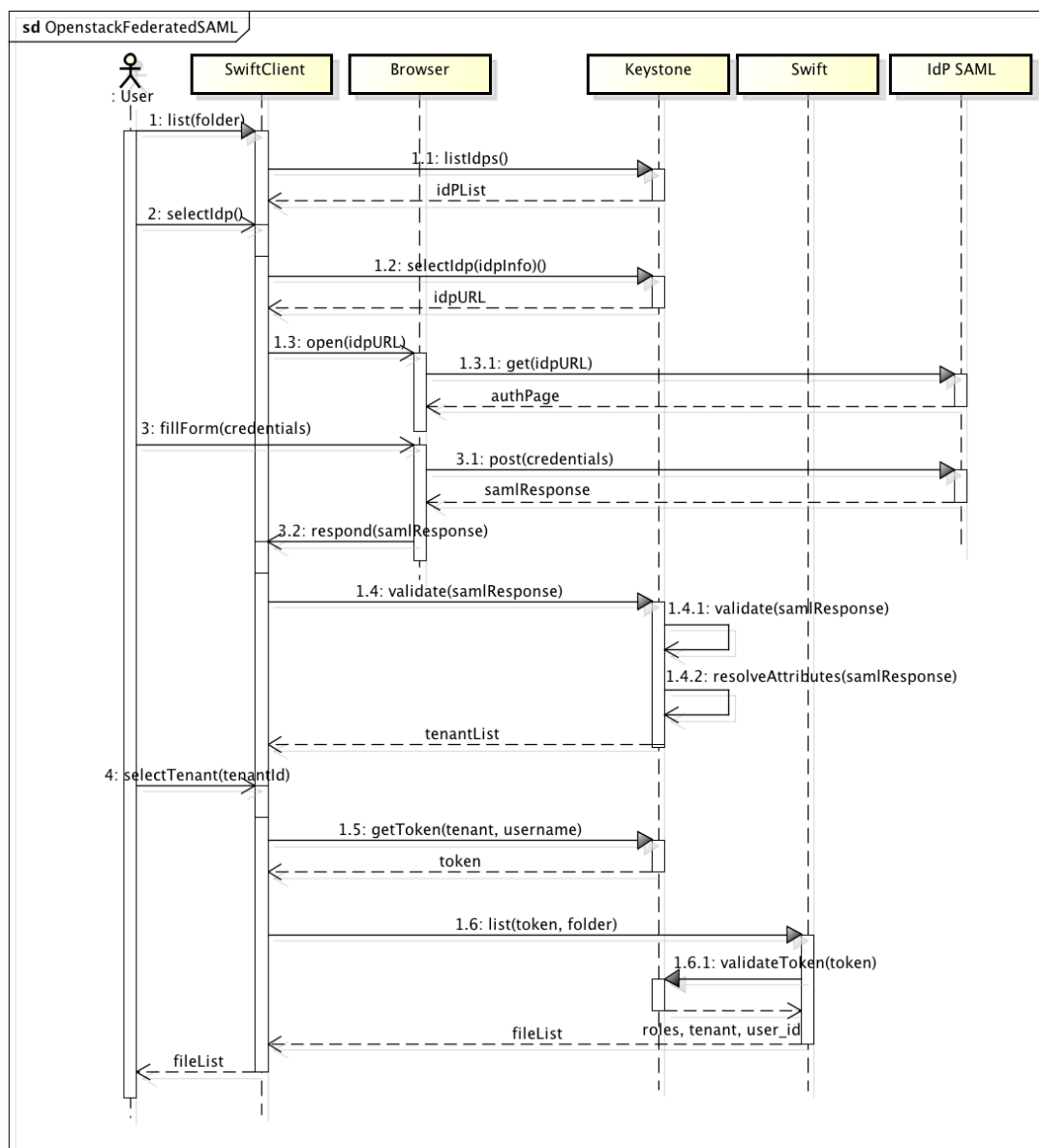


Figura 5. Fluxo de Autenticação do Openstack federado com protocolo SAML

O fluxo de autenticação do Openstack federado utilizando um IdP OpenID Connect com perfil de cliente básico é similar ao diagrama da Figura 5. A Figura 6 mostra apenas o

trecho onde ocorrem as diferenças. No passo 1.4, no lugar da mensagem SAMLResponse, o IdP OpenID Connect devolve um código (*code*) que identifica a autenticação (3.1). O SP possui credenciais (*clientId* e *clientSecret*) cadastradas previamente no IdP. No momento da validação, o Keystone solicita ao IdP um *token* para obter acesso aos dados do usuário, enviando suas credenciais e o código identificador da autenticação (1.4.1). Além do *token* de acesso, o Keystone recebe do IdP um *token* de identificação que serve para o SP garantir que está se comunicando com o IdP correto. O protocolo OpenID Connect define uma série de validações que o SP deve fazer sobre o *token* de identificação (1.4.2). Na sequência, o Keystone solicita ao IdP os atributos do usuário, passando o *token* de acesso recebido (1.4.3). Daí em diante, o fluxo continua de forma similar ao protocolo SAML.

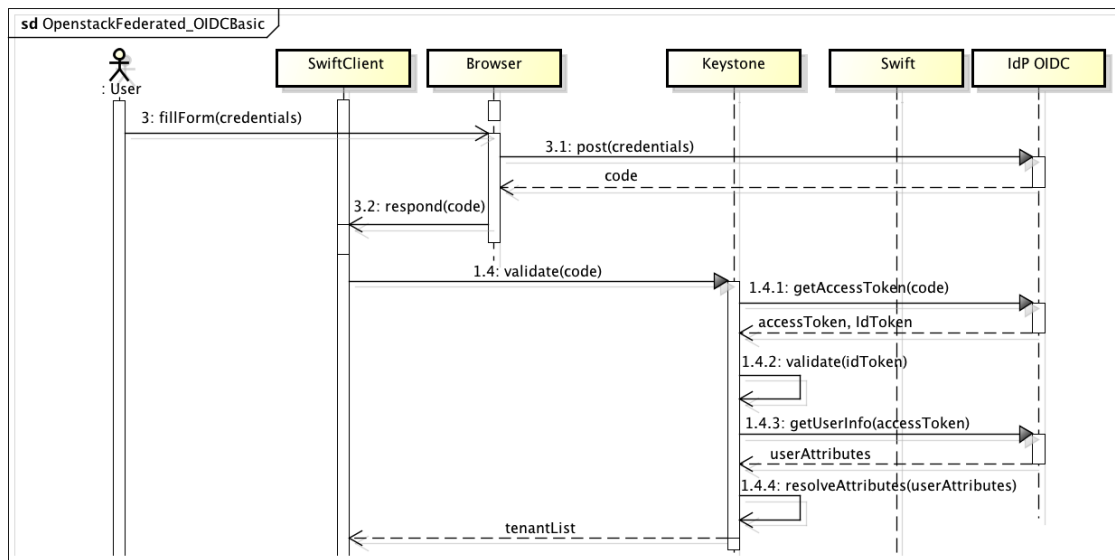


Figura 6. Autenticação do Openstack federado usando OpenID Connect Basic Profile

Para o Openstack federado utilizando um IdP OpenID Connect com perfil de cliente implícito, como fizemos no caso anterior, mostramos apenas o trecho com as diferenças na Figura 7. Em relação ao perfil de cliente básico, a diferença está no passo 1.4 que possui uma mensagem a menos. Desta vez, em vez de passar o código (*code*) para receber o *token* de acesso e *token* de identificação do IdP, o Keystone já recebe essas informações do “cliente swift”.

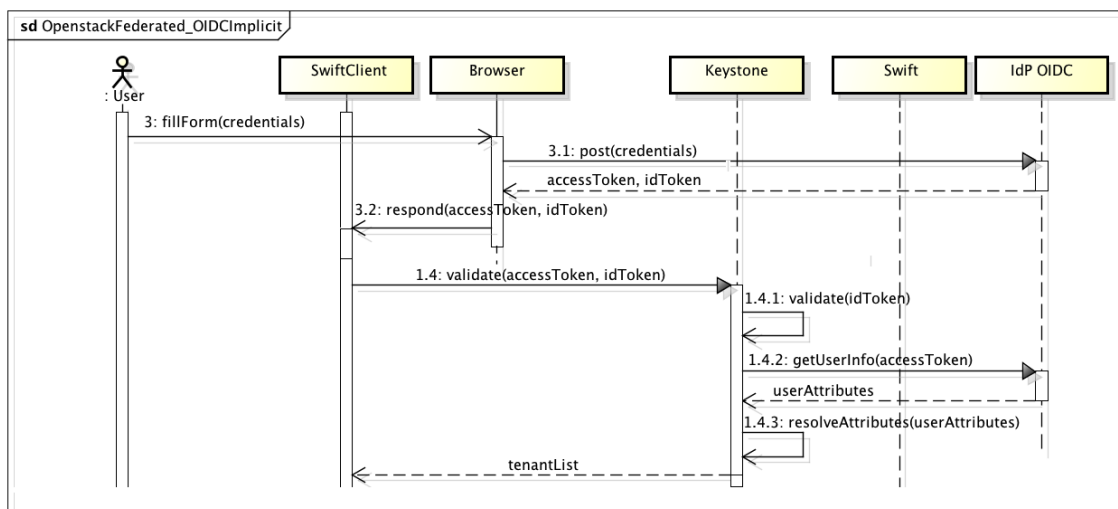


Figura 7. Autenticação do Openstack federado usando OpenID Connect Implicit Profile

6. Experimentos

Esta seção descreve os experimentos realizados para comparação de desempenho entre os protocolos SAML com *binding* HTTP Redirect e perfil *Web Browser SSO*, e OpenID Connect, nos perfis de usuário básico e implícito, integrados ao Openstack.

6.1. Ambiente

O ambiente usado para a realização do experimento é composto por um provedor de serviços de nuvem Openstack (SP), um IdP SAML e um IdP OpenID Connect. Os aplicativos usados pelo usuário (“cliente swift” e navegador) foram simulados a partir da ferramenta JMeter (<http://jmeter.apache.org>), executada no mesmo ambiente do SP.

O SP é composto por um computador desktop com 3GB de RAM, processador Athlon dual-core de 2 GHz, sistema Linux Ubuntu 12.04 LTS. Os seguintes módulos da plataforma Openstack foram instalados: Swift na versão Grizzly seguindo a configuração Swift All-In-One (http://docs.openstack.org/developer/swift/development_saio.html) e Keystone, na versão federada proposta por Chadwick (2013a). O computador está instalado no Centro de Informática (CIn) da UFPE, conectado à RNP através de um link de 10 Gbps.

O IdP SAML foi instalado em uma VM com 512 MB de RAM, processador Xeon de 2.4 GHz, sistema Linux Ubuntu 12.04 LTS. O sistema utilizado é o SimpleSAMLphp (<http://simplesamlphp.org>), configurado para funcionar com *binding* HTTP Redirect. O servidor pertence ao Grupo de Trabalho de Computação em Nuvem para Ciência (GT-CNC) da RNP e está no DIMAP/UFRN, também conectado à RNP através de link de 10 Gbps.

O IdP OpenID Connect foi instalado em uma VM com 1GB de RAM, processador Xeon de 2.4GHz, sistema Linux Ubuntu 12.04 LTS. O sistema usado é o MITREid Connect (<http://id.mitre.org/connect>), configurado para funcionar com perfis de usuário básico e implícito. O servidor faz parte do Laboratório de Gestão de Identidade (GID Lab) da RNP e está localizado no PoP-MA, conectado ao backbone da RNP através de link de 3Gbps.

6.2. Cenários de Teste

O Keystone foi configurado com dois projetos para usuários autenticados pelos IdP SAML e IdP OpenID Connect respectivamente. No projeto UFRN, os usuários com atributo “brEduAffiliationType” igual a “employee” possuem acesso irrestrito por possuírem papel *federated_admin*, enquanto os com atributo “student” apenas podem listar e receber arquivos, com papel *federated_member*. Já no projeto GIDLab, o usuário “josegidlab” possui o papel *federated_admin* e todos os outros, *federated_member*.

Para configurar este cenário, os passos descritos em [Siu 2012] e [Chadwick 2013b] foram seguidos. Eles servem para mapear atributos do IdP (id do usuário, email etc.) com os atributos do SP (papéis, projetos etc.).

O JMeter foi configurado para realizar 300 repetições sequenciais da listagem de uma pasta no Swift para cada IdP. Os tempos e a quantidade de bytes trafegados são medidos para cada um dos 6 passos definidos na Figura 3.

A comunicação entre o navegador e o IdP SAML é criptografada utilizando o protocolo HTTPS. Todas as demais conexões são realizadas com o protocolo HTTP, sem criptografia. A quantidade de bytes medida não contempla a criptografia quando o HTTP é usado.

Não consideramos no teste o tempo de autenticação nos IdPs. Para isto, os usuários são previamente autenticados nos dois IdPs e os *cookies* de sessão dos provedores são configurados no JMeter. Desta forma, os IdPs, através do mecanismo de SSO, retornam as mensagens de autenticação sem solicitar novamente as credenciais do usuário.

6.3. Resultados

Os tempos para realização de cada passo (1 a 6) da Figura 3 e a quantidade de bytes recebidos em cada passo são apresentados na Tabela 2 e discutidos na sequência. Em relação aos tempos medidos, observamos que os mesmos não apresentaram uma distribuição normal. Portanto, comparamos as medianas das amostras coletadas para os protocolos SAML e OpenID Connect utilizando o teste de hipótese não paramétrico de Kruskal-Wallis [Montgomery 2003 p.589-591], uma extensão do teste de Mann-Whitney-Wilcoxon [Montgomery 2003 p.585-588] para mais de duas amostras. Consideramos um intervalo de confiança (IC) de 95% nas análises.

Tabela 2. Tempo e quantidade de bytes recebidos em 300 repetições de chamadas ao servidor Swift para listar uma pasta vazia usando protocolos SAML e OpenID Connect

<i>Variável</i>	<i>Passo</i>	<i>SAML WebApp SSO profile</i>	<i>OpenID Connect Basic profile</i>	<i>OpenID Connect Implicit profile</i>	<i>p</i>
Tempo (em ms)	1	8 (7-8)	8 (7-8)	8 (7-8)	0,00
	2	20 (19-20)	13 (13-14)	13 (13-14)	0,00
	3	47 (46-48)	40 (38-55)	95,5 (77,75-171)	0,00
	4	674,5 (620-997,5)	1106 (886-1414)	821 (764-1260)	0,00
	5	73 (61-83,25)	75 (61-88)	75 (61-84)	0,14
	6	76 (67-83)	75,5 (68-83)	76 (69-83)	0,98
	Total	920 (854,8-1202)	1540 (1120-1656)	1172 (1061-1557)	0,00
Bytes recebidos	1	1553 (1553-1553)	1553 (1553-1553)	1554 (1554-1554)	0,00
	2	1271 (1265-1279)	256 (256-256)	285 (285-285)	0,00
	3	11930 (11930-11930)	316 (316-316)	1294 (1294-1294)	0,00
	4	992 (992-992)	968 (968-968)	968 (968-968)	0,00
	5	2610 (2610-2610)	2610 (2610-2610)	2609 (2609-2609)	0,00
	6	168 (168-168)	168 (168-168)	168 (168-168)	1
	Total	18530 (18520-18540)	5871 (5871-5871)	6878 (6878-6878)	0,00

O passo 1 representa a listagem dos IdPs disponíveis. Apesar dos testes de hipótese apontarem que as distribuições são diferentes ($p=0$), podemos observar que as medianas e intervalos interquartílicos dos tempos e da quantidade de bytes recebidos são bastante próximos. O passo 2 representa a seleção do IdP, onde os dados do IdP selecionado são retornados. No caso do SAML, os dados incluem a chave pública, o que faz com que sejam bem maiores comparados aos do OpenID Connect.

Nos passos 3 e 4, o cliente, que se comunicava com o SP (Keystone) nos passos anteriores, passa a se comunicar com o IdP. Como os IdPs estão fisicamente em localidades distintas e conectados com velocidades distintas (seção 6.1), medimos a latência entre o cliente e o SP, localizados na UFPE, e os IdPs SAML (UFRN) e OpenID Connect (PoP-MA), obtendo os resultados de 10ms e 32ms, respectivamente. Estes tempos foram descontados dos tempos medidos para cada chamada realizada aos IdPs.

O passo 3 representa a autenticação do usuário no IdP. O tempo maior do perfil implícito do OpenID Connect é justificável pelos dados retornados virem como fragmento, que precisam ser processados e uma requisição HTTP preparada para enviar os dados ao Keystone. Nos outros dois protocolos, os dados já vem formatados na URL de redirecionamento. O volume trafegado para o perfil implícito também é maior em relação ao perfil básico, uma vez que os *tokens* de autenticação vêm na resposta em lugar do identificador (*code*). O protocolo SAML é o que apresenta maior volume trafegado, uma vez que a mensagem no formato XML é maior, contendo chaves públicas, assinaturas e todos os dados de autenticação do usuário.

O passo 4 é o mais representativo em relação ao tempo total. Ele contempla a validação dos dados da autenticação, o mapeamento de atributos, e a listagem dos projetos associados ao usuário. As validações do protocolo SAML são computacionalmente

custosas, principalmente por incluir a validação da assinatura através de algoritmo RSA. Porém, o protocolo OpenID Connect apresenta um tempo maior devido às conexões entre SP e IdP, duas no perfil básico e uma no perfil implícito. O OpenID Connect, nos dois perfis, também realiza validações do *token* de identificação (*id token*) conforme explicado na seção 4.1. A resposta desta chamada é a lista de projetos disponíveis para o usuário, apresentando valores próximos apesar de estatisticamente diferentes.

A partir do passo 5, o cliente volta a falar com o SP (Keystone e Swift). As chamadas são as mesmas, independentemente do protocolo do IdP. O passo 5 representa a requisição do *token* Openstack, apresentando tempos e quantidades de bytes recebidos similares. O passo 6 representa a listagem de uma pasta contendo o mesmo arquivo, também apresentando tempos e bytes trafegados iguais ($p=0,98$ e $p=1$).

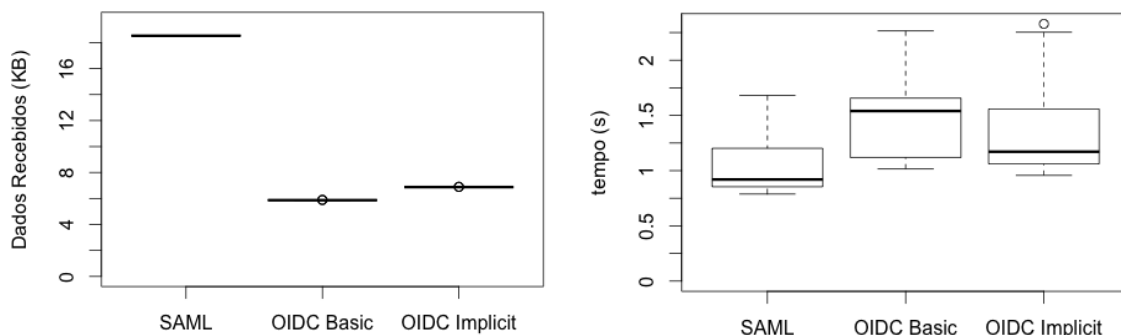


Figura 8. Quantidade de bytes recebidos pelo cliente (a) e tempo total (b) em experimentos com protocolos SAML e OpenID Connect (OIDC) nos perfis básico e implícito.

Podemos concluir que, no total dos passos, o protocolo SAML gera um maior volume de dados recebidos pelo cliente (18,5KB contra 5,9 e 6,9KB, respectivamente), mas realiza suas operações num tempo menor que o OpenID Connect (0,9s contra 1,5 e 1,2s), conforme visto na Figura 8. A diferença de tempo podia ser ainda maior em favor do SAML se o protocolo HTTPS fosse utilizado para as requisições ao IdP OpenID Connect. Entre os perfis básico e implícito do OpenID Connect, o implícito apresentou melhor desempenho com menor tempo e apenas 1KB de diferença. O tráfego entre SP e IdP existente no protocolo OpenID Connect não foi contabilizado, pois a medição foi realizada apenas nas interações entre cliente e provedores (SP e IdP). Se considerado, a diferença no total de tráfego gerado pelos protocolos diminuiria.

7. Conclusões e Trabalhos Futuros

Este trabalho considera que é vantajoso se integrar serviços, como plataformas de nuvem, a federações de identidades. Estas, possuem relacionamento próximo ao usuários, podendo fornecer informações importantes sobre eles.

Ao integrarmos a plataforma Openstack com federações através dos protocolos SAML e OpenID Connect, observamos que o SAML gera uma quantidade grande de informações no padrão XML, apesar dos experimentos utilizarem *binding HTTP Redirect* e perfil *Web Browser SSO*. A integração do SAML também exige maior esforço, por exemplo com a necessidade de geração e armazenamento de chaves criptográficas para assinatura das mensagens.

Da forma como foi configurado, o SAML comprova, entretanto, que o esforço para integração compensa, obtendo-se tempos finais menores devido à ausência de tráfego de rede entre IdP e SP. Além disso, o protocolo é mais maduro, suportando esquemas de atributos bem definidos no padrão X.500/LDAP.

A integração do Openstack com o protocolo OpenID Connect é simples, facilitada pelo protocolo em comum, o REST. Apesar de mais recente que o SAML, o OpenID Connect é utilizado por grandes provedores de serviços na Internet como Google, Paypal e Microsoft. O perfil de cliente implícito do OpenID Connect mostrou-se mais eficiente que o perfil básico, com tempo final próximo ao protocolo SAML. No entanto, sua implementação é mais complexa, necessitando de desenvolvimento no cliente (*browser*) para encaminhar os *tokens* ao SP. Ele também não elimina totalmente a comunicação entre SP e IdP.

Como extensões deste trabalho, serão consideradas uma análise de segurança, bem como a inclusão novos protocolos e padrões, por exemplo, o WS-Federation. Novos cenários também serão avaliados, como o uso de assinatura e criptografia das mensagens no protocolo OpenID Connect, a integração com a plataforma Shibboleth, possivelmente usando a solução de Diniz *et al* (2013) e a repetição dos testes usando o protocolo HTTPS no lugar do HTTP, tornando o cenário mais próximo de um caso real, onde a criptografia é necessária. Pretendemos ainda propor um modelo para autenticação e autorização de plataformas de nuvens através da integração com federações de identidade através de mecanismos como ABAC e UCON_{ABC} [Tavizi *et al* 2012; Lazouski *et al* 2012; Marcon Junior *et al* 2013].

Referências

- Chadwick, D. W. (2013) “Federated Keystone”, <http://wiki.openstack.org/wiki/Keystone/Federation/Blueprint>, April.
- Chadwick, D. W. (2013) “Handling ACLs that use UserIDs in Federated Keystone”, <https://blueprints.launchpad.net/keystone/+spec/acls-userids-federation>, April.
- Chadwick, D. W. and Fatema, K. (2012) “A privacy preserving authorisation system for the cloud”, In: *Journal of Computer and System Sciences*, i.78, p.1359-1373. Elsevier.
- Chadwick, D. W., Casenove, M. and Siu, K. (2013) “My private cloud – granting federated access to cloud resources” In: *Journal of Cloud Computing*, p.1-16. SpringerOpen.
- Columbus, L. (2013) “Gartner Predicts Infrastructure Services Will Accelerate Cloud Computing Growth”, <http://www.forbes.com/sites/louisacolumbus/2013/02/19/gartner-predicts-infrastructure-services-will-accelerate-cloud-computing-growth/>, February.
- Columbus, L. (2013) “Predicting Enterprise Cloud Computing Growth”, <http://www.forbes.com/sites/louisacolumbus/2013/09/04/predicting-enterprise-cloud-computing-growth/>, September.
- Diniz, T. F. S., Silva, C. E. and Araujo, R. (2013) “Integrando o Openstack Keystone com Federações de Identidades”, In: *XIII Simpósio Brasileiro em Segurança da Informação e de Sistemas Computacionais*, p.465-474. SBC.
- Eucalyptus (2013) “The Eucalyptus Cloud”, <http://www.eucalyptus.com/eucalyptus-cloud/iaas>, December.
- Eucalyptus (2013) “Eucalyptus 3.4.0 Administration Guide”, <http://www.eucalyptus.com/docs/eucalyptus/3.4/admin-guide-3.4.0.pdf>, p.38-45, November.
- IETF (2012) “The OAuth 2.0 Authorization Framework”, <http://tools.ietf.org/html/rfc6749>, October.

- Karp, A., Haury, H. and Davis, M. H. (2009) “From ABAC to ZBAC: The Evolution of Access Control Models”, <http://www.hpl.hp.com/techreports/2009/HPL-2009-30.pdf>, February.
- Khan, R. H., Ylitalo, J. and Ahmed, A. S. (2011) “OpenID authentication as a service in OpenStack”, In: *7th International Conference on Information Assurance and Security*, p.372-377. IEEE.
- Lazouski, A. et al. (2012) “Usage Control in Cloud Systems”, In: *The 7th International Conference for Internet Technology and Secured Transactions*, p.202-207. IEEE.
- Lynch, L. (2011) “Inside the Identity Management Game”, In: *IEEE Internet Computing*, v.15 (5), p.78-82. IEEE.
- Marcon Junior, A. L. et al. (2013) “A UCON ABC Resilient Authorization Evaluation for Cloud Computing”, In: *IEEE Transactions on Parallel and Distributed Systems*, p.1-11. IEEE.
- Montgomery, D. C. and Runger, G. C. (2003) “Applied Statistics and Probability for Engineers”, John Wiley & Sons, Inc., 3rd edition.
- OASIS (2005) “Bindings for the OASIS Security Assertion Markup Language (SAML) V2.0”, <http://docs.oasis-open.org/security/saml/v2.0/saml-bindings-2.0-os.pdf>, March.
- OASIS (2005) “Profiles for the OASIS Security Assertion Markup Language (SAML) V2.0”, <http://docs.oasis-open.org/security/saml/v2.0/saml-profiles-2.0-os.pdf>, March.
- Openstack (2013) “Administration Guide”, <http://docs.openstack.org/grizzly/openstack-compute/admin/bk-compute-adminguide-grizzly.pdf>, December.
- Revar, A. G. and Bhavsar, M. D. (2011) “Securing User Authentication using Single Sign-On in Cloud Computing”, In: *Nirma University International Conference on Engineering*, p.1-4. IEEE.
- Sakimura, N. et al. (2013) “OpenID Connect Basic Client Implementer's Guide 1.0 - draft 29”, http://openid.net/specs/openid-connect-basic-1_0.html, October.
- Sakimura, N. et al. (2013) “OpenID Connect Implicit Client Implementer's Guide 1.0 - draft 12”, http://openid.net/specs/openid-connect-implicit-1_0.html, October.
- Sette, I. S. and Ferraz, C. A. G. (2013) “Integrando Openstack com Provedores de Identidade OpenID Connect e SAML: Uma Análise Comparativa” In: *XIII Simpósio Brasileiro em Segurança da Informação e de Sistemas Computacionais*, p.497-506, SBC.
- Silva, L. M. et al. (2013) “Estudo de Caso: Integração de Clientes da Nuvem Openstack Swift Com Uma Federação de Identidade”, In: *XIII Simpósio Brasileiro em Segurança da Informação e de Sistemas Computacionais*, p.455-464. SBC.
- Siu, K. (2012) “A Role Mapping Service for the Keystone Identity Server”, <https://blueprints.launchpad.net/keystone/+spec/role-mapping-service-keystone>, November.
- Tavizi, T., Shajari, M. and Dodangeh, P. (2012) “A Usage Control Based Architecture for Cloud Environments”, In: *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*, p.1534-1539. IEEE.